A

需要记录一个跟乘积有关的状态。

发现可以用 $\gcd(k, \prod w)$ 来算，只和因数个数有关，所以可以过。

B

相当于每次加入一条路径，然后这条路径会对点 p 在 t 时刻贡献 $\dfrac{(t-dis(u,p)+l)}{l}$ 下取整。

那么剩下的贡献就和 $(t-dis(u,p)) \mod l$ 有关，那么首先把相同的 l 拿出来，由于 l 很小，增加可以跑暴力。

那么我们需要建 l^2 个数据结构，分别和 $dis(u,p), l$ 有关，剩下一个单点加链查可以差分为子树加单点查。

复杂度 $qd^2 \log n + kd \log n$。

C

若没有经过行头或尾，答案是显而易见的，先上下移动至目标行，在左右调整。

注意到每一次不是正常走都会经过第一列或最后一列，因此考虑维护 L_i, R_i 表示到达第 i 行的左端点和右端点的操作次数，转移两次即可。

这样，时间复杂度为 $O(n \log n)$，其中 $\log$ 为区间求最小值。

D

问题理解为计算 Bob 最多能拿走多少颗糖果。

分析 Bob 的具体路径：Bob 拿走一颗糖后要把它送到一个空地，且每次只能拿 1 颗。

那么我们不妨将 Bob 拆成若干个“任务”：从某个商店拿一颗糖果，送到某个空地，然后继续去下一家要偷的商店。

分析一下取糖果的顺序：由于 Alice 是从左到右走的，所以肯定是从左到右的取糖果。

然后分析一下每个任务的代价（即所需要走的路径长度），设商店 i 到最近的空地的距离为 d_i，那么似乎取一次糖果的代价为 $2d_i$。但是这真的对吗?

事实上，对于商店 i 取得最后一颗糖果，Bob 并不需要回到商店 i，而是可以去到一个右边空地后，直接走向下一个要取糖果得商店。

设商店 i 右边的空地为 s_i，商店 i 和空地之间最左的商店为 nxt_i。

如果先送到一个空地，然后在走到右边的商店，我们的代价为 $2s_i - nxt_i - i$。

而直接走过去，不添加糖果的代价 $nxt_i - i$。

同时这样做省去了走到下一个商店的代价，我们可以得出这个策略的代价为 $(2s_i - nxt_i - i) - (nxt_i - i) = 2s_i - 2nxt_i$。

故我们最后一颗糖果的代价为 $\min(2d_i, max(0, 2s_i - 2nxt_i))$。

有了这个性质后，我们可以发现每个商店的糖果实际上可以分成两种：

- 1 颗“最后糖果”：代价为 $\min(2d_i, max(0, 2s_i - 2nxt_i))$。
- $a_i - 1$ 颗“普通糖果：代价为 $2d_i$。

问题变成：从左到右选择尽量多的物品，使得 Bob 来得及在 Alice 到达对应位置前开始拿这颗糖果。

当处理到 i 时，设前面取糖果花费的代价为 tot（注意没有加上路径长度）。

那么只有 $tot \le (i-1)$ 时，Bob 才可以取糖果。这种问题是不是感觉很熟悉?

考虑反悔贪心，用一个大根堆存储之前取得所有糖果。

对于每个可选糖果，直接选；否则若对顶代价大于当前代价，则弹出对顶，然后看看能不能再塞一些糖果。

有个小细节，返回贪心的时候要先贪“普通糖果”，再贪“最后糖果”（不能反过来），不然就不符合上面的结论。